American University of Sharjah
College of Engineering

TinyML Project – Milestone 3 Report

On-Device Prototype and Optimization

Energy-Efficient Human Activity Recognition on the Edge

Team Member	AUS ID
Lee Ting Sen	B00103724
Khalifa Alshamsi	B00078654
Vineetha Addanki	G00111196

Project Track	Track B: UCI-HAR public benchmark with Arduino target-domain adaptation and live on-device evidence.
Board / Sensor	Arduino Nano 33 BLE Sense with onboard LSM9DS1 accelerometer and gyroscope.
Selected M3 Build	`m3_v2_1_mixed_6ch`: mixed UCI + Arduino DS-CNN, full-integer INT8, deployed through `model_data.h`.
Main Live Result	Right-pocket controlled live Serial: 0.9040 accuracy / 0.9089 macro F1. Left-pocket robustness live Serial: 0.8901 accuracy / 0.8826 macro F1.

May 2026

Executive Summary

Milestone 3 answers two questions: whether the human activity recognition model can run on an Arduino with live sensor input, and what had to change after Milestone 2 to make that deployment credible. The final evidence-collection build is `m3_v2_1_mixed_6ch`, a 6-channel depthwise-separable CNN trained from UCI-HAR source data plus Arduino V2/V2.1 target-domain adaptation data, then converted to a full-integer INT8 TFLite Micro C array. On the board, the sketch reads live LSM9DS1 accelerometer and gyroscope samples, forms 128-sample windows with 64-sample stride at 50 Hz, normalizes and quantizes inputs on-device, invokes TFLite Micro, and prints parseable Serial predictions.

The deployment evidence satisfies the main M3 requirements. The sketch compiles and runs on Arduino Nano 33 BLE Sense; the INT8 model is 10,288 bytes; flash usage is 177,504 bytes; RAM usage is 111,144 bytes; the tensor arena is 61,440 bytes; and average `Invoke()` latency is 34.113 ms over 50 calls. True live Serial testing achieved 0.9040 accuracy / 0.9089 macro F1 under the controlled right-pocket condition and 0.8901 accuracy / 0.8826 macro F1 under the left-pocket robustness condition. The main remaining failure is stair-direction confusion, especially `WALKING_UPSTAIRS -> WALKING_DOWNSTAIRS`. The model is therefore a working M3 prototype and evidence-collection build, not a fully robust final HAR system.

1 R1 Deployment Pipeline

1.1 System Path

The project remains Track B because UCI-HAR is the public source-domain benchmark and official held-out test reference. The Arduino datasets are used to adapt and validate the deployable model in the target hardware setting. To avoid overclaiming, this report separates four evidence roles: public benchmark, target-domain adaptation, raw replay validation, and true live Arduino Serial evaluation.

Table 1: Training and adaptation datasets used in Milestone 3.

Dataset	Role	Size / use
UCI-HAR v1.0	Public Track B source dataset and official benchmark.	The training split was used for source learning; the official test set stayed held out. Final split: 5,551 train windows, 1,801 validation windows, and 2,947 official test windows.
WISDM classic v1.1	Secondary inspection and domain-gap reference.	Not used for training because its label taxonomy does not match the six UCI-HAR classes.
Arduino V2	Corrected 50 Hz target-domain data for adaptation and grouped validation.	236 usable windows: 44 walking, 30 upstairs, 30 downstairs, 44 sitting, 44 standing, and 44 laying.
Arduino V2.1 long adaptation	Extra Arduino adaptation data for walking, sitting, standing, and laying.	Four 5-minute captures; 932 windows before capping and 720 windows after capping.

Table 2: Held-out validation and live evidence sets.

Evidence set	Role	Size / result
Right-pocket raw validation, `right_60s`	Held-out controlled raw sensor replay validation.	240 replay windows; used for candidate sanity checks, not final live accuracy.
Left-pocket raw validation, `left_30s`	Held-out robustness raw sensor replay validation.	115 replay windows; used to expose pocket placement and orientation weakness.
Right-pocket live Serial logs, `7_control_trials`	Final controlled on-device evidence.	125 scored Serial rows, 0.9040 accuracy, and 0.9089 macro F1.
Left-pocket live Serial logs, `8_robust_trials`	Final robustness on-device evidence.	91 scored Serial rows, 0.8901 accuracy, and 0.8826 macro F1.

The key evidence rule is separation. Training, normalization, and INT8 representative calibration use only UCI-HAR training data, the Arduino V2 training split, and capped Arduino V2.1 long-adaptation windows. UCI-HAR official test, Arduino V2 grouped validation, right/left raw validation, and all live Serial logs are excluded from training and calibration. The pipeline below shows what happens after the selected model is packaged for live Arduino inference.

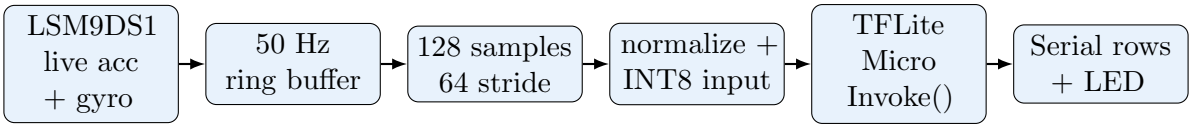

Figure 1: Live Arduino inference pipeline. Training uses TensorFlow/Keras, conversion uses full-integer INT8 TFLite, and deployment uses a TFLite Micro C array.

1.2 Model Conversion and Arduino Sketch

The deployment path turns the selected TensorFlow/Keras model into an Arduino-ready TFLite Micro artifact:

```
TensorFlow/Keras -> full-integer INT8 TFLite PTQ -> C array in model_data.h ->
                                TFLite Micro on Arduino
```

The selected model is packaged into `arduino/tinyml_har_m3/model_data.h`. The header stores the INT8 model bytes, input/output quantization parameters, class labels, and six-channel normalization constants. The Arduino sketch `arduino/tinyml_har_m3/tinyml_har_m3.ino` implements the live pipeline:

- reads accelerometer x/y/z and gyroscope x/y/z from LSM9DS1,
- targets 50 Hz sampling using scheduled timing,
- converts gyroscope values from degrees/second to radians/second,
- stores a 128-sample ring buffer and runs inference every 64 new samples,
- applies the exported training/adaptation normalization constants,
- quantizes normalized inputs using the TFLite input scale and zero point,
- calls `interpreter->Invoke()` and measures latency with `micros()`,
- prints timestamp, window id, prediction id, label, confidence, latency, top-1 score, top-2 label/score, and running average latency.

1.3 Quantization Package

The selected `m3_v2_1_mixed_6ch` model uses full-integer INT8 post-training quantization. Representative calibration windows are class-balanced from training-side data only: UCI-HAR train, Arduino V2 train, and capped V2.1 long-adaptation windows. The representative set excludes UCI-HAR test, Arduino V2 grouped validation, right-pocket raw validation, left-pocket raw validation, and all live Serial logs.

QAT was not used for M3 because INT8 PTQ did not materially reduce macro F1 on the selected candidate in the saved FP32-vs-INT8 summaries. The remaining live weakness is stair-direction confusion, which is a data/domain issue rather than a quantization failure.

Table 3: Arduino deployment environment.

Item	Version / description
Board	Arduino Nano 33 BLE Sense.
Sensor	Onboard LSM9DS1 accelerometer and gyroscope.
Arduino IDE	2.3.8.
Board package	Arduino Mbed OS Nano Boards 4.5.0.
Sensor library	Arduino_LSM9DS1 1.1.1.
TinyML library	Harvard_TinyMLx 1.2.4-Alpha.
Primary output	Serial monitor prediction rows; simple LED state as auxiliary feedback.

2 R2 Deployment Metrics

R2 reports only deployment measurements from the Arduino build and Serial evidence. Laptop replay scores are used later for model-selection context, but laptop timing is not counted as an Arduino hardware metric.

Table 4: Measured deployment metrics for the selected INT8 Arduino candidate.

Metric	Value	Evidence / method
INT8 `.tflite` model size	10,288 bytes / 10.05 KB	Selected deployment file `m3_v2_1_mixed_6ch_int8.tflite`.
`model_data.h` file size	65,933 bytes / 64.39 KB	Generated C array and constants.
Tensor arena size	61,440 bytes / 60.00 KB	Boot Serial metadata.
Flash usage	177,504 bytes / 173.34 KB / 18%	Arduino IDE compile output.
Global dynamic memory	111,144 bytes / 108.56 KB / 42%	Arduino IDE compile output.
Average `Invoke()` latency	34,113 us / 34.113 ms	Serial latency summary averaged over 50 inferences.
Stability	Passed, about 156 seconds	Returned stability video evidence.
Demo evidence	At least three proof clips plus Serial logs	Sitting, standing, walking proof clips and live trial logs.

The model fits comfortably within the Arduino Nano 33 BLE Sense memory limits. Inference is triggered once per completed stride window rather than at every raw sensor sample, so the 34.113 ms `Invoke()` time is acceptable for this 50 Hz, 128-sample-window HAR prototype.

3 R3 Optimization: From Public Accuracy to Arduino Deployment

3.1 Optimization Storyline

The M3 optimization did not follow a simple "make the M2 model smaller" path. The M2 lightweight DS-CNN was already small and strong on UCI-HAR, but the UCI-trained model failed when replayed on real Arduino V2 captures. The baseline predicted all 236 Arduino V2 replay windows as `LAYING`, giving 0.1864 accuracy and 0.0524 macro F1. A deeper replay check found the same collapse for FP32 and INT8, so the failure was not caused by quantization. The real problem was domain shift caused by hardware, orientation, pocket placement, clothing, and normalization mismatch.

For readability, the optimization story is organized as four decisions:

1. **Respond to professor feedback.** Test focal loss and a 10-channel gravity feature idea to address posture confusion and orientation sensitivity.
2. **Diagnose the deployment gap.** Use Arduino V2 replay to show that a UCI-only model does not transfer to the pocket-mounted Arduino setting.
3. **Add target-domain data.** Use Arduino V2 and capped V2.1 long-adaptation data for training, while keeping right-pocket, left-pocket, and live Serial evidence held out.
4. **Select and package the deployable model.** Choose the mixed 6-channel DS-CNN, quantize it with full-integer INT8 PTQ, regenerate `model_data.h`, and verify live Arduino behavior.

3.2 Professor-Feedback Probes

The first M3 improvement pass targeted the professor's M2 feedback. Focal loss was tested because stair and posture classes were unevenly difficult. A 10-channel gravity-feature version was tested because `SITTING` and `STANDING` depend strongly on board angle and gravity direction. These probes were useful, but neither became the final deployed build. Focal loss improved some robustness behavior but was not the best controlled right-pocket candidate. The 10-channel gravity model remains a strong M4 candidate, but it adds Arduino preprocessing complexity and did not beat the simpler 6-channel pipeline for the controlled demo.

3.3 Arduino Data Expansion

After the UCI-only collapse, the project moved from public-dataset optimization to target-domain adaptation. Arduino V2 provided corrected 50 Hz six-class target data and exposed the placement/orientation gap. Arduino V2.1 then added four longer adaptation recordings for `WALKING`, `SITTING`, `STANDING`, and `LAYING`. Because V2.1 did not include stair classes, its windows were capped so the four available classes would not overwhelm `WALKING_UPSTAIRS` and `WALKING_DOWNSTAIRS`. This gave the model more Arduino-domain posture and walking examples while preserving stair-class balance. Held-out `right_60s`, held-out `left_30s`, and all live Serial logs remained evaluation evidence only.

3.4 Candidate Search and Decision

Table 5: Final V2.1 model-search decisions.

Model family	Question answered	Decision
UCI-only baseline	Does public training transfer directly?	No. Arduino replay collapses, so public-dataset accuracy alone is not enough.
Arduino-only	Is target data alone enough?	No. It has too little target data and poor UCI retention.
UCI pretrain + V2.1 fine-tune	Does sequential transfer work?	No. Fine-tuning was unstable with the small target dataset.
Mixed 6-channel DS-CNN	Can source and target data be balanced?	Selected for the M3 controlled demo and Arduino deployment package.
Mixed focal 6-channel	Does focal loss help hard classes?	Useful, but weaker than the selected model for controlled right-pocket replay.
Mixed 10-channel gravity	Does explicit orientation help?	Useful M4 path, but more complex and weaker for the controlled right-pocket replay.

Table 6: Final V2.1 model-search metrics. Values are INT8 macro F1 where an INT8 candidate was evaluated.

Model family	UCI F1	V2 F1	Right F1	Left F1
UCI-only baseline	0.9025	0.0587	0.0526	0.0535
Arduino-only	0.1082	0.4686	0.8050	0.5094
UCI pretrain + V2.1 fine-tune	0.4630	0.1986	0.2582	0.3442
Mixed 6-channel DS-CNN	0.7960	0.7309	0.7741	0.3886
Mixed focal 6-channel	0.7396	0.7099	0.7357	0.5643
Mixed 10-channel gravity	0.8832	0.6189	0.5655	0.6856

The selected model is `m3_v2_1_mixed_6ch`. It keeps public UCI-HAR source knowledge, adapts to Arduino V2/V2.1, preserves the deployed six-channel Arduino feature path, and gives the best controlled right-pocket validation among the deployable candidates. The tradeoff is explicit: it sacrifices some UCI-HAR test performance and remains sensitive to left-pocket orientation, but it gives a working, measurable Arduino M3 prototype.

3.5 Required Before/After Evidence

Table 7: Before/after comparison between the M2 baseline and M3 improved candidate.

Metric	**M2 UCI-only DS-CNN**	**M3 V2.1 mixed 6-channel INT8**	**Change / tradeoff**
UCI-HAR test accuracy	0.9169	0.7984	Lower public-source score after adapting to Arduino data.
UCI-HAR test macro F1	0.9173	0.7960	Lower public-source score, accepted to improve target behavior.
Arduino V2 grouped macro F1	UCI-only replay collapsed	0.7309	Large target-domain improvement.
Held-out right_60s replay macro F1	Not measured for M2	0.7741	Strong controlled replay candidate.
Held-out left_30s replay macro F1	Not measured for M2	0.3886	Raw replay robustness gap.
True right-pocket live macro F1	Not measured for M2	0.9089	Live Arduino controlled evidence.
True left-pocket live macro F1	Not measured for M2	0.8826	Live Arduino robustness evidence.
Deployment TFLite size	13,460 bytes	10,288 bytes	INT8 reduces deployment model size.
Arduino latency / RAM	Not measured for M2	34.113 ms / 61,440-byte arena	Real deployment metrics added.

The result should be interpreted as a deployment optimization, not a claim that the M3 model is universally better. The M3 model is better for the target Arduino evidence-collection setting because it converts, fits, runs on hardware, and produces strong live Serial evidence under the validated right-pocket setup.

4 R4 On-Device Accuracy

After selecting and packaging the model, the next question is whether it works during live Arduino inference. The handout requires live Arduino predictions for on-device accuracy, so the table below separates public laptop benchmark scores, raw sensor replay validation, and true live Arduino Serial results.

Table 8: Offline benchmark, raw replay validation, and true live Arduino evaluation.

Evaluation setting	Acc.	Macro F1	Weighted F1	Evidence type
UCI-HAR official test, M2 baseline	0.9169	0.9173	0.9168	Laptop public benchmark.
UCI-HAR official test, M3 INT8 candidate	0.7984	0.7960	–	Laptop TFLite benchmark.
Arduino V2 grouped replay, M3 INT8	0.7573	0.7309	–	Held-out grouped raw sensor replay.
right_60s replay, M3 INT8	0.8708	0.7741	–	Held-out controlled raw sensor replay.
Right-pocket controlled live Serial	0.9040	0.9089	0.8999	Arduino live inference, 125 scored rows.

The right-pocket controlled live trial used at least 20 rows per class. Static classes were perfect in the returned logs. The only major error was `WALKING_UPSTAIRS -> WALKING_DOWNSTAIRS`.

Table 9: Right-pocket controlled live confusion matrix and per-class F1.

Actual \Predicted	Walk	Up	Down	Sit	Stand	Lay	F1
WALKING	20	0	0	0	0	0	1.0000
WALKING_UPSTAIRS	0	13	12	0	0	0	0.6842
WALKING_DOWNSTAIRS	0	0	20	0	0	0	0.7692
SITTING	0	0	0	20	0	0	1.0000
STANDING	0	0	0	0	20	0	1.0000
LAYING	0	0	0	0	0	20	1.0000

The controlled live score is higher than the public UCI-HAR score of the adapted model because the live trials were performed under the validated deployment protocol and fixed right-pocket placement. It is not a laptop replay score: it was computed from Arduino Serial prediction logs generated during live sensor inference.

5 R5 Robustness Test

R5 changes the placement condition. The controlled setup is right-pocket placement; the robustness setup is left-pocket placement, which changes both board orientation and movement coupling. This tests whether the live model remains stable when the Arduino is worn differently.

Table 10: Controlled and robustness comparison.

Condition	Acc.	Macro F1	Evidence type	Main failure mode
right_60s replay, INT8	0.8708	0.7741	Raw sensor replay	`WALKING_UPSTAIRS -> WALKING_DOWNSTAIRS`.
left_30s replay, INT8	0.4348	0.3886	Raw sensor replay	`SITTING -> LAYING`; `LAYING -> STANDING`.
Right-pocket live Serial	0.9040	0.9089	Arduino live inference, 125 rows	`WALKING_UPSTAIRS -> WALKING_DOWNSTAIRS`.
Left-pocket live Serial	0.8901	0.8826	Arduino live inference, 91 rows	`WALKING_UPSTAIRS -> WALKING_DOWNSTAIRS`.

Table 11: Left-pocket robustness live confusion matrix and per-class F1.

Actual \Predicted	Walk	Up	Down	Sit	Stand	Lay	F1
WALKING	10	0	0	0	0	0	0.9091
WALKING_UPSTAIRS	0	6	6	0	0	0	0.6667
WALKING_DOWNSTAIRS	2	0	18	0	2	0	0.7826
SITTING	0	0	0	22	0	0	1.0000
STANDING	0	0	0	0	15	0	0.9375
LAYING	0	0	0	0	0	10	1.0000

The live robustness result is much stronger than earlier left-pocket raw replay. This reinforces why the final M3 claim is based on live Arduino Serial predictions, while replay remains diagnostic context. The dominant live robustness issue remains stair-direction separation, not static posture separation.

5.1 Deployment Placement Guidance

Teammate testing showed that the model is sensitive to board placement and orientation. The recommended controlled setup is the right pocket. If left pocket is used, the board should be oriented to resemble right-pocket placement as closely as possible. Static postures depend strongly on gravity/board angle: standing worked best with the board facing downward at about 90 degrees, sitting with a diagonal angle around 135 degrees, and laying flatter/horizontal around 180 degrees. Walking should be straight and natural; excessive leg lifting can push predictions toward `WALKING_DOWNSTAIRS`. These are deployment findings, not hidden bugs, and they explain the remaining confusion matrices.

6 R6 Challenges and Lessons Learned

The main lesson from M3 is that deployment quality was limited more by data domain and placement protocol than by neural-network size. Three challenges shaped the final result.

Table 12: M3 challenges, responses, and remaining risk.

Challenge	What we did	Remaining lesson / risk
UCI-HAR to Arduino domain shift	Audited Arduino V2 replay, compared FP32 and INT8 behavior, and retrained with mixed UCI + Arduino data.	The UCI-only model collapsed to `LAYING`; Arduino V2 `acc_x` was about -3.40 standard deviations from the UCI-HAR training standardizer mean.
Evidence separation	Separated UCI benchmark, Arduino grouped replay, held-out raw replay, and live Serial predictions.	Only live Arduino Serial logs are on-device accuracy; replay is useful but should not be reported as live performance.
Placement robustness	Tested controlled right-pocket placement and left-pocket robustness, then documented board-angle guidance.	Static postures depend on gravity/board angle, and stair direction remains the hardest live class.

If starting again, the team would define fixed board orientation earlier, collect calibration and stair data before model selection, and keep right/left placement protocols consistent from the beginning.

7 R7 Plan for M4

For M4, the goal is to turn the M3 evidence-collection build into a more robust demonstration model. The plan follows directly from R6: collect the missing stair and placement data, reduce orientation sensitivity, and only add model complexity when it addresses a measured weakness.

Table 13: M4 improvement plan.

#	Planned action	Reason
1	Collect more balanced live upstairs and downstairs examples across both pockets.	Main live failure is stair-direction confusion.
2	Standardize board orientation before each run.	Placement sensitivity is the main practical limitation.
3	Add a short calibration pose or orientation-normalization step.	Helps align gravity direction across pockets and users.
4	Explore orientation-invariant features or gravity-frame alignment.	May reduce dependence on board angle.
5	Revisit the 10-channel gravity candidate.	It showed better left-pocket raw replay but requires a more complex Arduino path.
6	Test with more users, days, and environments.	Current evidence is strong for M3 but still limited.
7	Use QAT only if future PTQ becomes unstable.	Current INT8 PTQ does not materially degrade the selected model.

Repository Package

The final repository contains the Arduino sketch, `model_data.h`, selected INT8 model, training and packaging scripts, live scoring script, README instructions, report source/PDF, hardware metrics, live Serial confusion matrices, and the M3 submission checklist. The final report artifact is exported as `tinyml_milestone3.pdf`. Before final upload, attach the returned video evidence and Serial evidence required by the handout. If the page limit or video-length rule is enforced strictly, compress the report toward 3–5 pages and provide either a 30–90 second edited demo clip or the proof clips plus Serial logs as supporting evidence.

Final Review Highlights

This section is included to make final review easier before submission; remove it from the PDF if the final upload must be concise.

- Added a dataset and evidence register covering UCI-HAR, WISDM, Arduino V2, Arduino V2.1, right/left raw replay validation, and right/left live Serial evidence.
- Made the evidence-separation rule explicit: only training-side data is used for training, normalization, and representative INT8 calibration; held-out tests and live logs remain excluded.
- Added quantization package details: full-integer INT8 PTQ, training-side representative data, no QAT for M3, and the reason quantization is not the main remaining failure.
- Reorganized R3 into a clearer storyline: professor-feedback probes, Arduino domain-shift diagnosis, target-domain data expansion, candidate search, final selection, and then the required before/after evidence.
- Revised the full report flow without changing the R1–R7 structure: clearer executive summary, stronger transitions between deployment metrics and optimization, a smoother handoff into live accuracy and robustness, and a compact R6 challenge table.
- Relaxed the report layout for readability: changed to 11pt text, wider margins, more paragraph/table spacing, clearer page breaks, split the dataset register into two tables, and split the R3 model-search table into decision and metric tables.
- Corrected `model_data.h` size to 65,933 bytes / 64.39 KB to match the current README and artifact summary.
- Updated the closing repository/submission paragraph so it no longer says the report still needs to be exported, and added the remaining handout caveats for page count and video evidence.